

Token-Benchmark: Nutzung & Anleitung

Anleitung für Kollegen — Tool starten, Container, Grenzen

Datenquelle: benchmark-9a72151a.json

Datum: Juli 2026

Runs: 540 (n=10 je Kombination, ausser Real-Task n=5)

Harnesses: Pi 0.80.2 | Claude Code 2.1.170 / 2.1.196

Internes Dokument · SelectLine Software AG

1 Schnellstart — Web-UI

Die Web-UI ist die empfohlene Bedienoberfläche. Sie läuft lokal im Browser und braucht keine Installation außer Python 3.10+.

Repository: <https://github.com/Schnoitzel/token-benchmark>

Voraussetzung: Pi und Claude Code müssen auf dem Host eingeloggt sein (OAuth, s. Abschnitt 5).

```
# Im Repository-Verzeichnis:
python3 server.py

# Dann im Browser (Windows) öffnen:
http://localhost:8000
```

Was die UI kann

- Task + Modell + Anzahl Wiederholungen auswählen und Benchmark starten
- Ergebnisse live verfolgen (Token-Breakdown: input / cache↑write / cache↓read / output / Overhead / Total)
- Tabellen mit Median + min-max je Kombination (aus aggregates)
- Drill-down: Kosten-Breakdown, Methodik-Erklärung ('Wie erhoben?'), Provenienz
- Theme-Umschalter (dunkel/hell), Präsentations-Modus (3 Kernzahlen)
- Qualität bewerten (Button 'Qualität bewerten' → /api/judge, blinder LLM-Richter)
- Real-Task-Reset (Button 'Repo zurücksetzen' → git restore .)
- Letzte Suite laden und alle bisherigen Ergebnisse anzeigen

2 CLI-Nutzung

Alternative zur UI für schnelle, scriptbare Läufe.

Häufige Befehle

Trockenlauf (keine echten Kosten)

```
python3 main.py --dry-run
```

Overhead-Kernzahl messen (5 Wdh.)

```
python3 main.py --complexity baseline --repeat 5
```

Alle Tasks, Haiku, 3 Wdh.

```
python3 main.py --models 'Haiku 4.5' --repeat 3
```

Einzelner Task, alle Modelle

```
python3 main.py --tasks medium-bash --repeat 10
```

Bericht aus neuestem Lauf

```
python3 report.py
```

Blinde Qualitätsbewertung (Opus-Richter)

```
python3 judge.py
```

Alle Parameter (main.py --help)

Parameter	Bedeutung	Default
--complexity	Nur Tasks dieser Komplexität (baseline/trivial/simple/medium/complex/real/all)	all
--tasks	Kommagetrennte Task-IDs (z.B. baseline-overhead,trivial-fact)	alle

Parameter	Bedeutung	Default
--models	Kommagetrennte Modell-Labels (z.B. 'Haiku 4.5')	alle
--harnesses	Nur dieser Harness (pi / claude-code)	beide
--repeat	Wiederholungen je Kombination	1
--output	Pfad zur Ausgabe-JSON (default: results/benchmark-<id>.json)	auto
--dry-run	Keine echten API-Aufrufe, Dummy-Antworten	—

3 Modelle & Tasks anpassen

Modelle (models.py)

Modelle sind in models.py als Liste von Model-Objekten definiert. Jedes Modell hat label, pi_id (Pi-Flag --model) und cc_id (Claude Code --model). Zum Hinzufügen einfach neues Model(...) -Objekt in MODELS einhängen.

```
# Beispiel: neues Modell hinzufügen (models.py)
from models import Model, MODELS
# MODELS.append(Model("Haiku 3.5", "claude-haiku-3-5", "haiku-3-5"))
```

Tasks (tasks.py)

Tasks sind in tasks.py als Liste von Task-Objekten definiert. Pflichtfelder: id, complexity, prompt. Optional: use_tools=True, num_turns, repo_dir.

```
# Beispiel: neuen Task hinzufügen (tasks.py)
from tasks import Task, TASKS
# TASKS.append(Task(
#     id="mein-task",
#     complexity="simple",
#     prompt="Erkläre X in drei Sätzen.",
# ))
```

Wichtig: Synthetische Tasks dürfen keine Pfad-Abhängigkeiten enthalten (laufen in leerer Sandbox). Real-Tasks brauchen repo_dir.

Preise (pricing.py)

Alle Kosten werden einheitlich aus pricing.py berechnet. Bei Preisänderungen nur diese eine Datei aktualisieren.

4 Container-Setup (Docker)

Der Container ermöglicht faire Messungen (beide Harnesses nativ Linux) und einfache Distribution an Kollegen.

Voraussetzungen: Docker installiert, eigener Anthropic-OAuth-Zugang.

Schnellstart

```
# 1. Image bauen (einmalig, ~400 MB)
docker build -t token-benchmark .

# 2. Host-Credentials importieren (Pi + CC müssen auf dem Host eingeloggt sein)
./docker/run.sh import-creds
```

```
# 3. UI starten (http://localhost:8000)
./docker/run.sh ui

# 4. Oder: CLI-Benchmark
./docker/run.sh bench --tasks medium-bash --repeat 10
```

Alle run.sh-Befehle

Befehl	Was es tut
./docker/run.sh ui	Startet Web-UI auf Port 8000
./docker/run.sh bench [args]	Startet main.py mit beliebigen Parametern
./docker/run.sh import-creds	Importiert Pi + CC Credentials vom Host
./docker/run.sh login	Interaktiver Login im Container (headless)
./docker/run.sh shell	Bash im Container (für Debugging)

Hinweise zum Login

- Empfohlen: Host-Credentials importieren (./docker/run.sh import-creds). Voraussetzung: Pi (TUI: /login) und Claude Code (claude auth login) müssen vorher auf dem Host eingeloggt sein.
- Credentials werden in ein persistentes Docker-Volume (~/.pi, ~/.config/claude) gemountet und bleiben nach Container-Neustart erhalten.
- Jeder Kollege loggt sich selbst ein — kein Key-Weitergabe nötig.
- Real-Tasks werden im Container automatisch übersprungen (repo_dir nicht verfügbar).

Ergebnisse persistieren

Das results/-Verzeichnis wird vom Host gemountet (./results:/app/results). Ergebnisse landen direkt auf dem Host und bleiben nach Container-Stopp erhalten.

5 Bekannte Grenzen & Hinweise

Aspekt	Einschränkung
Real-Tasks	Werden im Container übersprungen (JavaFX-Repo nicht im Container). Nur auf dem Host mit repo_dir konfigurierbar.
Plattform-Asymmetrie (Host)	Auf dem Windows/WSL-Host laufen Pi (Linux) und CC (Windows-Binary) in verschiedenen OS. Betrifft Tool-Tasks, NICHT den Single-Turn-Overhead (~8x).
OAuth-Login	Das Tool nutzt ausschließlich OAuth (kein API-Key). Jeder Nutzer braucht ein aktives Anthropic-Abo (Claude Pro / Team).
Cache-TTL	5-Minuten-TTL für Prompt-Cache angenommen (claude-3-Doku). Für Claude-4-Modelle nicht explizit verifiziert.
Qualitätsbewertung	judge.py verbraucht Opus 4.8 Tokens (teuer). Nur bei Bedarf ausführen. Bei Opus-Aufgaben wird Sonnet 4.6 als Richter verwendet.
Preise	Stand Juni 2026. Bei Änderungen pricing.py aktualisieren.